AI 388 / CS 388 Natural Language Processing

Module 2: Multiclass and Neural Classification

Topics 5, 6, 8, 9 (+ initialization in Topic 7)

Fall 2026 midterm study guide

Built from the lecture videos/transcripts, slides, Canvas exercises, Assignments 1–3 and the Fall 2022/2023 midterms.

Prepared October 2026 · lecture notation throughout

How to use this guide

Each **Topic** number is the item number on the official Fall 2026 midterm topic list. Section numbers are *topic.section*, so "2.4" is Topic 2, Section 4. Every topic has the same layout:

- **In plain words**: the idea in one paragraph.
- **Key boxes**: definitions and formulas in **lecture notation**.
- **Steps**: the algorithm or exam procedure, step by step.
- **Worked examples**: full calculations (checked numerically with Python).
- **Past exam**: every relevant Fall 2022/2023 midterm question and Canvas exercise, with the official answer and an explanation. Official errata are noted.
- **Traps**: places where convention differences or common slips cost points.
- **Practice**: original questions, with answers right after.

Notation (lecture): input $\mathbf{x}$; feature vector $f(\mathbf{x}) \in \mathbb{R}^d$; weights $\mathbf{w}$; label $y \in \{-1, +1\}$ (binary) or $y \in \mathcal{Y}$ (multiclass); step size α; D training examples $(\mathbf{x}^{(i)}, y^{(i)})$; log is the natural log unless a question says otherwise. Perceptron decision rule: positive iff $\mathbf{w}^\top f(\mathbf{x}) > 0$ (a score of 0 is negative). Where a past exam used a different convention, the box says so in red.

Exam reminders. Auto-graded short answers must match the requested format exactly (rounding, fractions vs. decimals, list syntax like `[w1, w2]`). Adding explanation text makes the answer wrong. Long answers: show work concisely for partial credit and box final answers.

Contents

How to use this guide **1**

5 Multiclass Classification: Weights and Features **3**

5.1 Setup . . . 3

5.2 Example multiclass tasks (lecture 2.0.2) . . . 4

5.3 Practice . . . 4

6 Multiclass Perceptron (and Multiclass Logistic Regression) **6**

6.1 Multiclass perceptron . . . 6

6.2 Multiclass logistic regression (softmax regression) . . . 7

6.3 Practice . . . 8

7 Optimization: SGD, Step Size, Initialization **9**

7.1 The optimization problem . . . 9

7.2 Step size: the quadratic example . . . 9

7.3 Choosing step sizes (lecture recipe) . . . 10

7.4 Regularization (lecture: "don't really use") . . . 11

7.5 Initialization (neural networks) . . . 11

7.6 Practice . . . 12

8 Feedforward Neural Networks: Definition and Initialization **13**

8.1 Definition . . . 13

8.2 Why it works: the XOR example (lecture 2.1.0) . . . 14

8.3 Parameter counting and shape checking . . . 14

8.4 Initialization . . . 15

8.5 Practice . . . 15

9 Training Neural Networks **17**

9.1 Objective . . . 17

9.2 Backpropagation . . . 17

9.3 Implementation in PyTorch (lecture 2.2.1, A2) . . . 18

9.4 Practical training techniques (lecture 2.2.2) . . . 19

9.5 Practice . . . 19

TOPIC 5

Multiclass Classification: Weights and Features

Sources: Lectures 2.0.0 Multiclass Classification, 2.0.2 Multiclass Classification Examples; slides seg-9, seg-11; Exercise 2.2; Fall 2023 Q1(2).

In plain words

With more than two labels, we can't just ask "is the score above 0?". Instead we give every class its own score and predict the class with the **highest** score (argmax). There are two equivalent bookkeeping schemes. Either each class has its *own weight vector* ("different weights", DW, used for neural nets), or there is one big weight vector and the *features depend on the hypothesized class* ("different features", DF, used for structured prediction).

5.1 Setup

- Label set (output space) $\mathcal{Y}$, e.g. {health, sports, science}.
- **One-vs-all** (one binary classifier per class) is a possible hack, but it can fail. With 3 classes in a line (1s, then 2s, then 3s), class 2 can't be separated from {1,3} linearly. So we use genuinely multiclass models instead.

Two equivalent formulations

Different weights (DW): one weight vector per class.

$$\hat{y} = \arg\max_{y \in \mathcal{Y}} \ \mathbf{w}_y^\top f(\mathbf{x}) \qquad (f(\mathbf{x}) \in \mathbb{R}^n, \ \ |\mathcal{Y}| \text{ weight vectors } \mathbf{w}_y \in \mathbb{R}^n)$$

Different features (DF): one weight vector; features conjoin the input with a *hypothesized* label y (not the gold label):

$$\hat{y} = \arg\max_{y \in \mathcal{Y}} \ \mathbf{w}^\top f(\mathbf{x}, y) \qquad (f(\mathbf{x}, y) \in \mathbb{R}^{n|\mathcal{Y}|})$$

Standard construction: $f(\mathbf{x}, y)$ is $f(\mathbf{x})$ copied into the block for class y, with zeros elsewhere. Each feature then means "sentence contains word j **and** the label is y". $\mathbf{w}$ is all the $\mathbf{w}_y$ concatenated, and $\mathbf{w}^\top f(\mathbf{x}, y) = \mathbf{w}_y^\top f(\mathbf{x})$.

- DW is natural for neural nets: compute a representation of $\mathbf{x}$ once, then score all classes at the output layer. DF would require rerunning the network for every y.
- DF is essential for structured prediction (tag sequences, trees) where $\mathcal{Y}$ is huge and features must look at properties of both input and output (Topic 20).
- Stack the DW vectors as rows of a matrix $W \in \mathbb{R}^{|\mathcal{Y}| \times n}$: then all scores are $Wf(\mathbf{x})$ (Topic 9).

Worked example: topic classification (lecture)

$\mathbf{x}$ = "too many drug trials, too few patients". Features [drug, patients, baseball]: $f(\mathbf{x}) = [1, 1, 0]$. **DW:** Suppose (made-up) $\mathbf{w}_{\text{health}}^\top f(\mathbf{x}) = 7.6$ and $\mathbf{w}_{\text{sports}}^\top f(\mathbf{x}) = -1.9$, plus a science score. Predict the argmax, **health**.

DF: $f(\mathbf{x}, \text{health}) = [1,1,0\,|\,0,0,0\,|\,0,0,0]$, $f(\mathbf{x}, \text{sports}) = [0,0,0\,|\,1,1,0\,|\,0,0,0]$, $f(\mathbf{x}, \text{science}) = [0,0,0\,|\,0,0,0\,|\,1,1,0]$. With $\mathbf{w} = [\,\mathbf{w}_{\text{health}}\,|\,\mathbf{w}_{\text{sports}}\,|\,\mathbf{w}_{\text{science}}\,]$ the scores are identical to DW.

5.2 Example multiclass tasks (lecture 2.0.2)

- **Text/topic classification** (news, newsgroups): a common benchmark, often artificial.
- **Textual entailment (NLI):** 3 classes for a premise/hypothesis pair: *entailment*, *contradiction*, *neutral*. Hard for BoW; little progress until neural nets.
- **Entity linking/disambiguation:** map the mention "Armstrong" to a Wikipedia article (about 4.5M classes). Needs features that relate the mention and its context to the candidate article.
- **Authorship attribution:** stop-word frequencies and style matter. Twitter study (Schwartz et al.): SVM with character 4-grams + word 2–5-grams, about 70% accuracy on 50 authors with 1,000 tweets each. Distinctive "k-signatures" (n-grams unique to an author).

Midterm Fall 2023 Q1(2)

Input text $\mathbf{x}$, output one of five labels. Most correct name?

Official answer: A. Classification. (Not language modeling, clustering, or parsing.)

Canvas Exercise 2.2

Q1 binary sentiment: all of binary perceptron, binary LR, multiclass perceptron, multiclass LR can be used. Binary is a special case of multiclass. Q2 five-way fine-grained sentiment with a single model:

Answer: multiclass perceptron, multiclass LR (binary methods can't). Q3 want examples "more correct" (large margin, not just barely right):

Answer: logistic regression (keeps updating). Q4 want a concrete "end goal" where training stops updating once the data are separated:

Answer: perceptron.

Traps

- In DF, y in $f(\mathbf{x}, y)$ is the *hypothesized* label being scored, not the gold label. No cheating.
- DF feature dimension $= n \times |\mathcal{Y}|$. The DW parameter count is the same: $|\mathcal{Y}|$ vectors of length n.
- Multiclass prediction is an argmax, not a comparison with 0. Tie-breaking must be specified (Exercise 2.0: ties go to class 3).

5.3 Practice

Practice

1. $f(\mathbf{x}) = [2,0,1]$; $\mathbf{w}_A = [1,1,1]$, $\mathbf{w}_B = [0,3,2]$, $\mathbf{w}_C = [1,-1,0]$. Predicted class?
2. Write $f(\mathbf{x}, B)$ in DF form for Q1 (order A, B, C blocks), and the DF $\mathbf{w}$.
3. 10,000-word vocabulary, 20 topics. How many parameters in DW multiclass LR (no bias)?

Answers

1. Scores A 3, B 2, C 2, so **A**. 2. $f(\mathbf{x}, B) = [\,0,0,0,2,0,1,0,0,0\,]$; $\mathbf{w} = [\,1,1,1,0,3,2,1,-1,0\,]$ (score 2, matching). 3. $20 \times 10{,}000 = 200{,}000$.

TOPIC 6

Multiclass Perceptron (and Multiclass Logistic Regression)

Sources: Lecture 2.0.1 Multiclass Perceptron and Logistic Regression; slides seg-10; Exercises 2.0, 2.1, 2.2.

In plain words

Predict the highest-scoring class. If it's wrong, there are two classes to fix: **push down** the class you wrongly predicted (subtract the features from its weights) and **push up** the correct class (add the features to its weights). The other classes are untouched. Multiclass LR does a "soft" version: every class gets pushed down in proportion to its probability, and the correct class gets pushed up by $(1 - P(\text{correct}))$.

6.1 Multiclass perceptron

Multiclass perceptron (DW form)

for each epoch, **for** each $(\mathbf{x}^{(i)}, y^{(i)})$:

1. $y_{\text{pred}} = \arg\max_{y \in \mathcal{Y}} \mathbf{w}_y^\top f(\mathbf{x}^{(i)})$ (use the stated tie rule).
2. **if** $y_{\text{pred}} \neq y^{(i)}$: $\quad \mathbf{w}_{y_{\text{pred}}} \leftarrow \mathbf{w}_{y_{\text{pred}}} - \alpha f(\mathbf{x}^{(i)}), \qquad \mathbf{w}_{y^{(i)}} \leftarrow \mathbf{w}_{y^{(i)}} + \alpha f(\mathbf{x}^{(i)})$.

DF form: $\mathbf{w} \leftarrow \mathbf{w} + \alpha f(\mathbf{x}^{(i)}, y^{(i)}) - \alpha f(\mathbf{x}^{(i)}, y_{\text{pred}})$. If $y_{\text{pred}} = y^{(i)}$ the two terms cancel, so the "if" is technically unnecessary.

Binary is a special case: with two classes the update pushes $\mathbf{w}_+$ and $\mathbf{w}_-$ in opposite directions. Their difference $\mathbf{w}_+ - \mathbf{w}_-$ follows the binary perceptron (step 2α).

Worked example: lecture / Exercise 2.0 (ties go to class 3 = science)

Features [drug, patients, baseball]. Ex 1: "too many drug trials, too few patients" [1,1,0], gold 1 (health). Ex 2: "baseball players taking drugs" [1,0,1], gold 2 (sports). $\alpha = 1$, all weights 0.

step	scores (h, sp, sc)	pred	gold	weights after: $\mathbf{w}_h$ | $\mathbf{w}_{sp}$ | $\mathbf{w}_{sc}$
ex 1	(0, 0, 0)	3 (tie)	1	[1,1,0] | [0,0,0] | [-1,-1,0]
ex 2	(1, 0, −1)	1	2	[0,1,-1] | [1,0,1] | [-1,-1,0]

Answer: $\mathbf{w}_h = [0, 1, -1]$, $\mathbf{w}_{sp} = [1, 0, 1]$, $\mathbf{w}_{sc} = [-1, -1, 0]$. Exactly two updates, each touching two classes.

6.2 Multiclass logistic regression (softmax regression)

Model

$$P(y = \hat{y} \mid \mathbf{x}) = \frac{\exp\left(\mathbf{w}^\top f(\mathbf{x}, \hat{y})\right)}{\sum_{y' \in \mathcal{Y}} \exp\left(\mathbf{w}^\top f(\mathbf{x}, y')\right)} \qquad \text{(DW: } \exp(\mathbf{w}_{\hat{y}}^\top f(\mathbf{x})) / \textstyle\sum_{y'} \exp(\mathbf{w}_{y'}^\top f(\mathbf{x}))\text{)}$$

softmax: exponentiate the scores, then normalize by their sum. In matrix form, $P(\mathbf{y}|\mathbf{x}) = \text{softmax}(Wf(\mathbf{x}))$.
Binary LR is the 2-class case where the negative class has an implicit zero feature vector: its score is $e^{\mathbf{w}^\top \mathbf{0}} = 1$, which gives $e^z/(1+e^z)$.

Training: NLL and gradient

Loss $\mathcal{L} = -\log P(y^{(i)}|\mathbf{x}^{(i)}) = -\mathbf{w}^\top f(\mathbf{x}, y^{(i)}) + \log \sum_{y'} \exp(\mathbf{w}^\top f(\mathbf{x}, y'))$.

$$\nabla_{\mathbf{w}} \mathcal{L} = -f(\mathbf{x}, y^{(i)}) + \sum_{y'} P(y'|\mathbf{x})\, f(\mathbf{x}, y') \quad \Longleftrightarrow \quad \text{DW: } \nabla_{\mathbf{w}_y} \mathcal{L} = \left(P(y|\mathbf{x}) - \mathbb{1}[y = y^{(i)}]\right) f(\mathbf{x}).$$

SGD ($\mathbf{w} \leftarrow \mathbf{w} - \alpha\nabla$): the gold class gets $+\alpha(1 - P(y^{(i)}|\mathbf{x}))f(\mathbf{x})$; every other class y gets $-\alpha P(y|\mathbf{x})f(\mathbf{x})$.

Intuition (lecture): if $P(y^{(i)}|\mathbf{x}) \approx 1$ the gradient is ≈ 0. If all mass is on one wrong class y_{bad}, the update is $+\alpha f(\mathbf{x}, y^{(i)}) - \alpha f(\mathbf{x}, y_{\text{bad}})$, exactly the multiclass perceptron. In between it's a soft version, kicking every class down a bit.

Worked example: Exercise 2.1 (multiclass LR, step size 1)

$f(\mathbf{x}) = [1, 1, 0]$, gold class 1, all weights 0.

1. Scores all 0, so $P = \text{softmax}(0, 0, 0) = (1/3, 1/3, 1/3)$.
2. Gold: $\mathbf{w}_1 \leftarrow \mathbf{w}_1 + (1 - \frac{1}{3})f(\mathbf{x}) = [2/3, 2/3, 0]$. Others: $\mathbf{w}_2, \mathbf{w}_3 \leftarrow -\frac{1}{3}f(\mathbf{x}) = [-1/3, -1/3, 0]$.
3. Q3: training on this example repeatedly, **LR keeps updating** (probability never reaches exactly 1) while the **perceptron stops** once it's correct.

Check: the updates across classes sum to zero ($\frac{2}{3} - \frac{1}{3} - \frac{1}{3} = 0$), since $\sum_y (P(y) - \mathbb{1}[y = y^{(i)}]) = 0$.

Calculator recipe: softmax

Scores $s_1..s_k$. (1) Optional: subtract max s for stability (doesn't change the result). (2) e^{s_j} for each. (3) Divide each by the sum. Example: $(2, 1, 0) \to (7.389, 2.718, 1)/11.107 = (0.6652, 0.2447, 0.0900)$.

Traps

- The multiclass perceptron updates **two** weight vectors on a mistake; multiclass LR updates **all** of them on every example.
- Don't update the wrongly predicted class *up*: it's minus for y_{pred} and plus for the gold class.
- Ties: follow the stated rule exactly. At the first step every score is 0.
- The LR update for non-gold classes is $-\alpha P(y|\mathbf{x})f(\mathbf{x})$, not $-\alpha(1 - P)f(\mathbf{x})$.

6.3 Practice

Practice

1. Classes A, B, C; ties go to the alphabetically first class. Features [f1, f2]. Data: [1,0] gold B; [0,1] gold C; [1,1] gold A. One epoch of multiclass perceptron from zeros.
2. $f(\mathbf{x}) = [1, 2]$, $\mathbf{w}_1 = [1, 0]$, $\mathbf{w}_2 = [0, 1]$, $\mathbf{w}_3 = [0, 0]$, gold class 3, $\alpha = 0.5$. Compute P, the NLL, and the multiclass LR update.

Answers

1. Ex1: scores (0,0,0), pred A, wrong: $\mathbf{w}_A = [-1, 0]$, $\mathbf{w}_B = [1, 0]$. Ex2: scores (0,0,0), pred A, wrong: $\mathbf{w}_A = [-1, -1]$, $\mathbf{w}_C = [0, 1]$. Ex3: scores $(-2, 1, 1)$, tie B/C, pred B, wrong: $\mathbf{w}_B = [0, -1]$, $\mathbf{w}_A = [0, 0]$. Final $\mathbf{w}_A = [0, 0]$, $\mathbf{w}_B = [0, -1]$, $\mathbf{w}_C = [0, 1]$.
2. Scores (1, 2, 0): $P = (0.2447, 0.6652, 0.0900)$; NLL $= -\ln 0.0900 = 2.4076$. Updates: $\mathbf{w}_1 \leftarrow \mathbf{w}_1 - 0.5(0.2447) f(\mathbf{x}) = [0.8776, -0.2447]$; $\mathbf{w}_2 \leftarrow \mathbf{w}_2 - 0.5(0.6652) f(\mathbf{x}) = [-0.3326, 0.3348]$; $\mathbf{w}_3 \leftarrow [0, 0] + 0.5(0.9100) f(\mathbf{x}) = [0.4550, 0.9100]$.

TOPIC 7

Optimization: SGD, Step Size, Initialization

Sources: Lectures 1.1.2 Basics of Learning, Gradient Descent, 1.4.1 Optimization Basics, 2.2.1 Neural Net Implementation, 2.2.2 Neural Net Training, Optimization; slides seg-3, seg-8, seg-15, seg-16; Exercises 2.3a, 2.3b; A2 Part 1 (Q1); Fall 2023 Q1(1).

In plain words

Training means searching for weights that make the total loss small. SGD repeatedly picks one example (or a batch), computes which direction makes the loss go *up* (the gradient), and takes a step of size α the other way. Too small a step is slow. Too large a step overshoots, oscillates or diverges. For linear models the loss is convex, so where you start doesn't matter. For neural networks it's non-convex, so the starting point (initialization) matters a lot. In particular, all-zero initialization breaks learning.

7.1 The optimization problem

Objective and SGD (lecture)

$$\min_{\mathbf{w}} \ \mathcal{L}(\mathbf{w}) = \sum_{i=1}^{D} \mathcal{L}(\mathbf{x}^{(i)}, y^{(i)}, \mathbf{w}) \qquad \text{(training data fixed; } \mathbf{w} \text{ varies)}$$

SGD: repeatedly pick an example i and set $\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{x}^{(i)}, y^{(i)}, \mathbf{w})$.
The objective is a *sum over examples* (linear over examples). That's why one example's gradient is a cheap, noisy estimate of the full gradient.

- **Batch/full gradient descent:** sum the gradient over all D examples, then take one step. **Minibatch SGD:** average or sum over a batch (Topic 9).
- **Instances:** perceptron (perceptron loss), logistic regression (NLL), neural networks (NLL through backprop).
- **Convexity:** perceptron loss and LR's NLL are convex in $\mathbf{w}$ (with fixed features). Neural network objectives are **non-convex**.

7.2 Step size: the quadratic example

Worked example: lecture 1.4.1: $\mathcal{L}(w) = w^2$, start at $w = -1$

Gradient $\nabla \mathcal{L} = 2w$, so the update is $w \leftarrow w - \alpha \cdot 2w = (1 - 2\alpha)\, w$.

α	sequence $w_0, w_1, \ldots$	behaviour
0.1	$-1, -0.8, -0.64, -0.512, \ldots$	slow, monotone convergence
0.25	$-1, -0.5, -0.25, -0.125, \ldots$	faster
0.5	$-1,\ \mathbf{0}, 0, 0$	**one step to the optimum**
0.75	$-1, 0.5, -0.25, 0.125, \ldots$	oscillating, still converging
1	$-1, 1, -1, 1, \ldots$	**oscillates forever** (lecture)
1.1	$-1, 1.2, -1.44, 1.728, \ldots$	diverges

General rule for this function (derived): converges iff $|1 - 2\alpha| < 1 \iff 0 < \alpha < 1$; monotone for $\alpha < \frac{1}{2}$; exact in one step at $\alpha = \frac{1}{2}$.

Worked example: Assignment 2 Q1: $f(x_1, x_2) = (x_1 - 1)^2 + 8(x_2 - 1)^2$, start at the origin

1. Gradient: $\nabla f = \big(2(x_1 - 1),\ 16(x_2 - 1)\big)$. Optimum $(1, 1)$.
2. Per-coordinate error after one step: $(x_1 - 1) \leftarrow (1 - 2\alpha)(x_1 - 1)$ and $(x_2 - 1) \leftarrow (1 - 16\alpha)(x_2 - 1)$.
3. Convergence needs *both* $|1 - 2\alpha| < 1$ and $|1 - 16\alpha| < 1$, so $0 < \alpha < \frac{1}{8}$. The steep coordinate (x_2) sets the tipping point: at $\alpha = 1/8$, x_2 oscillates; above it, SGD diverges (e.g. `-lr 1`).
4. Distance after k steps: $\sqrt{(1 - 2\alpha)^{2k} + (1 - 16\alpha)^{2k}}$. One choice is $\alpha \approx 0.1066$, which reaches distance $0.0959 < 0.1$ in **10** steps. No constant α does it in 9 (the best possible after 9 steps is 0.1233).
5. Lesson: **one global step size has to suit every direction**. A flat direction wants a large step and a steep direction a small one. This motivates adaptive methods and Newton's method.

7.3 Choosing step sizes (lecture recipe)

Practical step-size choices

1. **Try them out** over orders of magnitude: $1, 10^{-1}, 10^{-2}, \ldots$ Neural models often need small ones (10^{-4}, 10^{-5}).
2. **Large → small schedules:** $\alpha_t \propto 1/t$ or $1/\sqrt{t}$ for epoch t (fixed schedule).
3. **Decrease on plateau:** reduce α when held-out (dev) performance stagnates.

Second-order and adaptive methods

Newton's method: $\mathbf{w} \leftarrow \mathbf{w} - \left(\nabla^2_{\mathbf{w}} \mathcal{L}\right)^{-1} \nabla_{\mathbf{w}} \mathcal{L}$, using the inverse Hessian (curvature). On a quadratic it jumps straight to the optimum. But the Hessian is **quadratic in the number of features**, so it's infeasible for big models.

Adagrad, Adadelta, Adam: "adaptive" methods, cheap *approximations* to the inverse Hessian that are **linear** in the number of parameters. They rescale the step per coordinate. A rare feature (someone's name) and a common one (*said*) shouldn't share one step size.

Adam (Kingma & Ba 2015) = adaptive per-coordinate step size + **momentum** (an exponentially weighted average of past gradients). It is the default for NLP in this course. Wilson et al. (NeurIPS 2017): adaptive methods can fit training data well but generalize worse at test time than carefully tuned SGD.

7.4 Regularization (lecture: "don't really use")

Classical statistics says don't fully optimize the training loss (bias–variance tradeoff), and add explicit regularization. In this course we usually don't add a regularizer. Instead we rely on **early stopping**, **imperfect optimization**, and tricks like **dropout**.

7.5 Initialization (neural networks)

Why initialization matters

1. **Can't use zeros for parameters that produce hidden layers.** With $\mathbf{z} = g(Vf(\mathbf{x}))$ and $V = 0$, $\mathbf{z} = g(\mathbf{0}) = \mathbf{0}$ for tanh and ReLU. The output layer sees all zeros, every hidden value and gradient is 0, and nothing ever changes. Nonzero random values also **break symmetry** between hidden units.
2. **Don't initialize too large:** tanh/sigmoid units **saturate**. Inputs like ± 5 sit on the flat part of the curve, so the gradient passed back is tiny and learning stalls. ReLU has an analogous problem: if a unit's input is always negative, it outputs 0 with gradient 0 (it's "stuck").
3. **Glorot/Xavier initializer:** $U\left[-\sqrt{\frac{6}{\text{fan-in}+\text{fan-out}}}, +\sqrt{\frac{6}{\text{fan-in}+\text{fan-out}}}\right]$ (fan-in/fan-out = number of inputs/outputs of the layer). Goal: keep the variance of activations and gradients about the same across layers, so one step size works everywhere.
4. **Batch normalization** (Ioffe & Szegedy 2015): during training, shift and rescale each layer's activations to mean 0, variance 1 over the batch. Useful for very deep nets, mostly in vision.

Linear models vs. NNs: zero initialization is fine for perceptron/LR (convex, every weight gets its own gradient from the features). For NNs it's fatal, and different random initializations can converge to different solutions.

Canvas Exercise 2.3a

Q1: Network $\mathbf{y} = \text{softmax}(Wg(Vf(\mathbf{x})))$ with all layers initialized to zero.

Answer: It won't learn; the gradients are all 0. Q2: What can improve performance or learning speed?

Answer: Adam (momentum-based) instead of SGD; Xavier initialization; batching. *Not* very large random initial values (saturation).

Canvas Exercise 2.3b (XOR with the PyTorch FFNN example)

Q1: Run 5–10 times on the XOR data.

Answer: Sometimes fits, inconsistent across random initializations (non-convex). Q2: Remove the example $[1, 1]$.

Answer: Fits perfectly (the remaining 3 points are linearly separable, an easy problem). Q3: Zero initialization instead of Xavier.

Answer: Always fails to fit.

Midterm Fall 2023 Q1(1), revisited

"LR with BoW features is convex" is true. "SGD can optimize LR" is true. "The perceptron is SGD on some loss" is true. Neural networks are non-convex, which is why initialization matters for them but not for LR.

Calculator recipe: gradient-descent steps on a quadratic

For $f(\mathbf{x}) = \sum_j a_j(x_j - c_j)^2$: error in coordinate j multiplies by $(1 - 2a_j\alpha)$ each step. After k steps the error is $(1 - 2a_j\alpha)^k\times$ the initial error. Converges iff $\alpha < 1/a_{\max}$.

Traps

- The SGD update *subtracts* the gradient (it minimizes loss). If a question maximizes log-likelihood, gradient *ascent* adds it. Same step.
- "Oscillates" (stays bounded, $|1 - 2a\alpha| = 1$) is different from "diverges" ($|1 - 2a\alpha| > 1$).
- Zero initialization: fatal for hidden layers, harmless for linear models. Large initialization: saturation.
- Newton's method is quadratic in the number of features; Adam etc. are linear.
- With α constant and $\mathbf{w}_0 = 0$, the perceptron is α-invariant. LR and NNs are not.

7.6 Practice

Practice

1. $\mathcal{L}(w) = 3w^2$, $w_0 = 2$. Give w_1, w_2 for $\alpha = 0.1$. Which α reaches 0 in one step? For which α does it diverge?
2. $f(x_1, x_2) = x_1^2 + 4x_2^2$, start $(1, 1)$, $\alpha = 0.2$. Two steps of GD. Converging?
3. A hidden layer uses ReLU and every weight is initialized to -10 with nonnegative BoW inputs. What happens?
4. Name the two ideas Adam combines. Which method uses the inverse Hessian, and why isn't it used for big networks?

Answers

1. $w \leftarrow (1 - 6\alpha)w$: $w_1 = 0.8$, $w_2 = 0.32$. One step at $\alpha = 1/6$. Diverges for $\alpha > 1/3$ (oscillates forever at exactly $1/3$).
2. $x_1 \leftarrow (1 - 0.4)x_1 = 0.6x_1$, $x_2 \leftarrow (1 - 1.6)x_2 = -0.6x_2$: $(0.6, -0.6)$, then $(0.36, 0.36)$. Converging, with x_2 oscillating in sign.
3. Every pre-activation is ≤ 0, so ReLU outputs 0 with zero gradient. The units are dead and nothing learns through them.
4. Adaptive per-coordinate step size + momentum. Newton's method; the Hessian is quadratic in the number of parameters.

TOPIC 8

Feedforward Neural Networks: Definition and Initialization

Sources: Lectures 2.1.0 Neural Networks, 2.1.1 Neural Network Visualization, 2.2.0 Feedforward Neural Networks, Backpropagation, 2.2.2 Neural Net Training, Optimization; slides seg-12, seg-13, seg-14, seg-16; Exercises 1.1 Q3, 2.3a, 2.3b; Fall 2022 Q1(6), Q3(c).

In plain words

A linear classifier draws one flat boundary in the original feature space. A feedforward network first **transforms** the features into a new "latent" space: multiply by a matrix (warp), add a bias (shift), and apply a nonlinearity (squash). Then it puts a linear classifier on top. In the transformed space, data that wasn't linearly separable (XOR, good/bad/not good/not bad) can become separable. The transformation is *learned* end to end instead of hand-designing conjunction features.

8.1 Definition

One-hidden-layer FFNN (lecture notation)

$$\mathbf{z} = g(Vf(\mathbf{x})), \qquad P(\mathbf{y} \mid \mathbf{x}) = \text{softmax}(W\mathbf{z}) = \text{softmax}(W\,g(Vf(\mathbf{x})))$$

symbol	shape	meaning
$f(\mathbf{x})$	n	input features (BoW, averaged embeddings, ...)
V	$d \times n$	first-layer weights (warps the space); bias optional ($Vf(\mathbf{x}) + \mathbf{b}$)
g	elementwise	nonlinearity: tanh, ReLU, sigmoid, ...
$\mathbf{z}$	d	hidden units = learned latent features
W	$\lvert\mathcal{Y}\rvert \times d$	output layer; row y = class y's weight vector (DW)
$P(\mathbf{y}\lvert\mathbf{x})$	$\lvert\mathcal{Y}\rvert$	vector of class probabilities

Without the hidden layer, $\text{softmax}(Wf(\mathbf{x}))$ is exactly **multiclass logistic regression** (Topic 6). An FFNN is multiclass LR on learned features $\mathbf{z}$.

8.1.1 Nonlinearities

name	formula	range	note
tanh	$\frac{e^a - e^{-a}}{e^a + e^{-a}}$	$(-1, 1)$	saturates at both ends; $\tanh'(a) = 1 - \tanh^2(a)$
ReLU	$\max(0, a)$	$[0, \infty)$	"hinge"; gradient 0 for $a < 0$, 1 for $a > 0$
sigmoid	$1/(1 + e^{-a})$	$(0, 1)$	$\sigma' = \sigma(1 - \sigma)$

Why a nonlinearity is required: without g, $W(Vf(\mathbf{x})) = (WV)f(\mathbf{x})$ collapses to a single linear map,

which is still a linear classifier. *Which* nonlinearity you use matters much less than using one.

8.2 Why it works: the XOR example (lecture 2.1.0)

Worked example: latent space makes XOR separable

Points (x_1, x_2): $(0,0)-$, $(1,0)+$, $(0,1)+$, $(1,1)-$. Not linearly separable. Take $V = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$, $g = \tanh$, so $\mathbf{z} = \big(\tanh x_1,\ \tanh x_2,\ \tanh(x_1 + x_2)\big)$. The lecture idealizes $\tanh(1) \approx \tanh(2) \approx 1$, so $\mathbf{z}$: $(0,0,0)-$, $(1,0,1)+$, $(0,1,1)+$, $(1,1,1)-$. The third unit behaves like an OR gate. Now $\mathbf{w} = [-1, -1, 1.5]$ separates them: scores $0,\ 0.5,\ 0.5,\ -0.5$ (with the > 0 rule, 0 is negative). With true tanh values (0.7616, 0.9640) the scores are $0,\ 0.381,\ 0.381,\ -0.077$, which still separates.

Visualization (2.1.1, Chris Olah): the three stages are *warp* (V), *shift* (bias), *nonlinear squash* (tanh into $[-1,1]^2$). Stacking layers can untangle two interleaved spirals. Deep nets = iterated transformations, all learned by backprop.

8.2.1 Deeper networks

$\mathbf{z}_1 = g(V_1 f(\mathbf{x}))$, $\mathbf{z}_2 = g(V_2 \mathbf{z}_1)$, …, $P(\mathbf{y}|\mathbf{x}) = \text{softmax}(W\mathbf{z}_k)$. Each matrix's input dimension must equal the previous layer's output dimension. **Universal approximation:** a network with enough hidden units can represent essentially any function, so any finite dataset without identical inputs that have different labels can be fit (Fall 2022 Q3(c)).

8.3 Parameter counting and shape checking

Counting parameters

Linear layer $\mathbb{R}^a \to \mathbb{R}^b$: ab weights ($+b$ biases if used). Nonlinearities and softmax have no parameters. One hidden layer: $dn + |\mathcal{Y}|d$ (no bias) or $dn + d + |\mathcal{Y}|d + |\mathcal{Y}|$ (with bias).

Worked example: counts

BoW $n = 10{,}000$, $d = 100$, 2 classes: $100 \cdot 10{,}000 + 2 \cdot 100 = 1{,}000{,}200$ weights (1,000,302 with biases). 300-dim input, $d = 200$, 2 classes: $60{,}000 + 400 = 60{,}400$ (60,602 with biases).

Midterm Fall 2022 Q1(6)

Use word embeddings + an FFNN to predict binary sentiment. Best implementation? ([in × out]) A. 300-d emb →[300×2]→ReLU→[2×2]→softmax B. 50-d emb →[50×75]→Tanh→[75×2]→log-softmax C. 300-d →[50×100]… D. 50-d →[50×150]→[150×2]→softmax E. 300-d→[300×200]→Tanh→[200×2]→Tanh→softmax.

Official answer: B.

Why: A: a 2-unit hidden layer is a severe bottleneck. C: dimension mismatch (300 into a 50-input layer). D: no nonlinearity between the linear layers, so it's still linear. E: a tanh *before softmax* squashes the logits to $[-1,1]$, so the model can never put probability near 1 on the right class. B:

dimensions chain, has a nonlinearity, and log-softmax pairs with NLL loss.

Canvas Exercise 1.1 Q3

Perceptron fails on good/bad/not good/not bad with the same features. Fix by changing the model?

Answer: Use a nonlinear model such as an FFNN. Not LR (still linear), not more epochs.

8.4 Initialization

See Topic 7, Section 7.5 for the full treatment. Summary:

1. **Never all zeros** for layers feeding hidden units: $\mathbf{z} = g(\mathbf{0}) = \mathbf{0}$, all gradients 0, nothing learns, and there is no symmetry breaking (Exercise 2.3a Q1, 2.3b Q3).
2. **Not too large:** tanh/sigmoid saturate, so gradients vanish. ReLU units can die if their inputs are always negative.
3. **Xavier/Glorot:** $U[-\sqrt{6/(\text{fan-in} + \text{fan-out})}, +\sqrt{6/(\text{fan-in} + \text{fan-out})}]$ keeps the variance of activations and gradients similar across layers.
4. Non-convex, so different random seeds can reach different solutions (XOR sometimes fails, Exercise 2.3b Q1).

Traps

- V is $d \times n$ (output $\times$ input) in lecture math. PyTorch's `nn.Linear(in, out)` lists input first. Exam questions write [in $\times$ out].
- No nonlinearity means a linear model, however many layers.
- Don't put tanh/sigmoid right before the softmax (Fall 2022 Q1(6) E).
- Softmax output with an NLL loss needs a log; log-softmax + NLLLoss is equivalent to raw logits + CrossEntropyLoss.
- Biases: say whether you count them in parameter totals.

8.5 Practice

Practice

1. Input 50-d averaged embeddings, hidden layers of 64 and 32 (ReLU), 5 classes. Give all weight shapes in lecture (out $\times$ in) form and the parameter count with biases.
2. True/False: a 3-layer network with identity activations can solve XOR.
3. With ReLU instead of tanh in the XOR example (same V, no bias), compute $\mathbf{z}$ for the four points. Is there a separating $\mathbf{w}$? If not, fix the third hidden unit with a bias.

Answers

1. $V_1 : 64 \times 50$, $V_2 : 32 \times 64$, $W : 5 \times 32$. Params $= (3200 + 64) + (2048 + 32) + (160 + 5) = 5{,}509$.
2. False: it collapses to one linear map.
3. $\mathbf{z}$: $(0,0,0)$, $(1,0,1)$, $(0,1,1)$, $(1,1,2)$. **Not separable**: on nonnegative inputs ReLU is the identity, so $z_3 = z_1 + z_2$ and the network is still linear here. Fix: $z_3 = \text{ReLU}(x_1 + x_2 - 1)$ gives

$(0,0,0), (1,0,0), (0,1,0), (1,1,1)$, and $\mathbf{w} = [1, 1, -2]$ scores $0, 1, 1, 0$, which is correct under the > 0 rule. The nonlinearity must actually "bend" somewhere the data lives.

TOPIC 9

Training Neural Networks

Sources: Lectures 2.2.0 Feedforward Neural Networks, Backpropagation, 2.2.1 Neural Net Implementation, 2.2.2 Neural Net Training, Optimization; slides seg-14, seg-15, seg-16; Exercises 2.3a/b; A2 Part 2.

In plain words

Same recipe as logistic regression: maximize the log-probability of the correct labels with (stochastic) gradient descent. The new part is getting gradients for the inner layers. **Backpropagation** is the chain rule applied backwards: compute the output error, pass an "error signal" back to the hidden layer, and use it to get gradients for V. In practice PyTorch's automatic differentiation does this for you. You write the forward pass, call `loss.backward()`, then `optimizer.step()`.

9.1 Objective

NLL for an FFNN (lecture form)

Let i^* be the index of the gold label and $\mathbf{e}_{i^*}$ the one-hot selector vector (e.g. $\mathbf{e}_2 = [0,1,0]$).

$$\log P(y = i^* | \mathbf{x}) = \mathbf{e}_{i^*} \cdot \log \text{softmax}(W\mathbf{z}) = \mathbf{w}_{i^*}^\top \mathbf{z} - \log \sum_j \exp(\mathbf{w}_j^\top \mathbf{z}), \qquad \mathcal{L} = -\log P(y = i^* | \mathbf{x}).$$

This is the multiclass LR objective with features $\mathbf{z}$, so the gradient for W is the multiclass LR gradient with $\mathbf{z}$ in place of $f(\mathbf{x})$.

9.2 Backpropagation

Gradients for $\mathbf{z} = g(\mathbf{a})$, $\mathbf{a} = Vf(\mathbf{x})$, $\mathbf{p} = \text{softmax}(W\mathbf{z})$

$$\begin{aligned}
\frac{\partial \mathcal{L}}{\partial W} &= (\mathbf{p} - \mathbf{e}_{i^*})\, \mathbf{z}^\top && \text{(output layer = multiclass LR)} \\
\delta_z = \frac{\partial \mathcal{L}}{\partial \mathbf{z}} &= W^\top (\mathbf{p} - \mathbf{e}_{i^*}) && \text{("error signal" passed back)} \\
\frac{\partial \mathcal{L}}{\partial \mathbf{a}} &= \delta_z \odot g'(\mathbf{a}) && (\tanh' = 1 - \mathbf{z}^2;\ \text{ReLU}' = \mathbb{1}[a > 0]) \\
\frac{\partial \mathcal{L}}{\partial V} &= \frac{\partial \mathcal{L}}{\partial \mathbf{a}}\, f(\mathbf{x})^\top && \text{(chain rule: } \partial\mathcal{L}/\partial\mathbf{z} \cdot \partial\mathbf{z}/\partial V)
\end{aligned}$$

Pattern (lecture): each parameter's gradient = (error signal coming back from above) $\times$ (input to that layer, $f(\mathbf{x})$ or $\mathbf{z}$), saved from the forward pass. The lecture sketches this; you aren't expected to derive deep backprop, but this structure is the whole idea.

Worked example: one forward/backward/SGD step (verified by finite differences)

$f(\mathbf{x}) = [1,0,1]$, $V = \begin{bmatrix} 0.5 & -1 & 0.2 \\ 0 & 1 & -1 \end{bmatrix}$, $g = \tanh$, $W = \begin{bmatrix} 1 & -1 \\ 0.5 & 2 \end{bmatrix}$, gold class 1, $\alpha = 0.1$.

1. $\mathbf{a} = Vf(\mathbf{x}) = [0.7, -1.0]$, $\mathbf{z} = \tanh \mathbf{a} = [0.60437, -0.76159]$.
2. Scores $W\mathbf{z} = [1.36596, -1.22100]$, $\mathbf{p} = [0.93002, 0.06998]$, $\mathcal{L} = -\ln 0.93002 = 0.07255$.
3. $\mathbf{p} - \mathbf{e}_1 = [-0.06998, 0.06998]$; $\partial\mathcal{L}/\partial W = \begin{bmatrix} -0.04229 & 0.05330 \\ 0.04229 & -0.05330 \end{bmatrix}$.
4. $\delta_z = W^\top(\mathbf{p} - \mathbf{e}_1) = [-0.03499, 0.20995]$; $\odot(1 - \mathbf{z}^2) = [-0.02221, 0.08817]$.
5. $\partial\mathcal{L}/\partial V = \begin{bmatrix} -0.02221 & 0 & -0.02221 \\ 0.08817 & 0 & 0.08817 \end{bmatrix}$. The middle column is 0 because $f_2 = 0$.
6. After the step $\theta \leftarrow \theta - 0.1\nabla$, the loss drops to 0.07003.

9.3 Implementation in PyTorch (lecture 2.2.1, A2)

```
class FFNN(nn.Module):
    def __init__(self, inp, hid, out):
        super().__init__()
        self.V = nn.Linear(inp, hid); self.g = nn.Tanh()
        self.W = nn.Linear(hid, out); self.softmax = nn.Softmax(dim=0)
        nn.init.xavier_uniform_(self.V.weight)   # nonzero init is critical
    def forward(self, x):
        return self.softmax(self.W(self.g(self.V(x))))

ffnn = FFNN(inp, hid, out)
optimizer = optim.Adam(ffnn.parameters(), lr=lr)
for epoch in range(num_epochs):
    for (inp, gold_label) in data:          # gold_label one-hot
        ffnn.zero_grad()                     # clear cached gradients!
        probs = ffnn.forward(inp)
        loss = torch.neg(torch.log(probs)).dot(gold_label)   # NLL
        loss.backward()                      # autodiff computes all gradients
        optimizer.step()                     # Adam update
    # check dev-set performance each epoch
```

- **Computation graph / autodiff:** write forward with differentiable torch operations, and `backward` is generated automatically. Converting to a raw Python number or numpy mid-network **breaks** gradient tracking.
- `zero_grad()` before each backward. PyTorch *accumulates* gradients in place, so forgetting it is a silent bug (A2 handout calls this the "most unintuitive piece").
- Losses: `NLLLoss` expects **log-probabilities**; `CrossEntropyLoss` expects **raw scores (logits)**. Check what yours expects (A2 handout).
- A2's DAN returns a log-probability distribution, uses NLLLoss + Adam, and checks dev accuracy each epoch.

9.4 Practical training techniques (lecture 2.2.2)

Checklist

- **Batching:** process B examples at once. Inputs become [batch, features] matrices, `forward` is unchanged, and probabilities become [batch, classes]. **Sum (or average) the loss over the batch** into one number. Faster through parallel matrix ops, and it can help optimization. Typical sizes 1–100 (16–32 common on GPU). Variable-length sentences are **padded** (A2: PAD index 0, `padding_idx=0`).
- **Initialization:** nonzero, appropriately scaled (Xavier) (Topic 7).
- **Dropout:** randomly zero parts of the network during training; use the full network at test time. It prevents co-adapted features (e.g. two correlated features with weights $+10$ and -9 fitting one odd example). Think of it as an ensemble of subnetworks. Too high a rate hurts; gains are modest.
- **Optimizer:** Adam (adaptive step + momentum) is the default. Tuned SGD may generalize better (Wilson et al. 2017).
- **Gradient clipping:** cap the gradient magnitude to avoid blow-ups.
- **Monitor:** print training loss each epoch and check dev accuracy to catch overfitting. NaNs usually mean the step size is too large ($\log 0$) (A2 tip).

Canvas Exercise 2.3a Q2

What improves performance or learning speed of $\text{softmax}(Wg(Vf(\mathbf{x})))$?

Answer: Adam, Xavier initialization, batching. Not very large random initial values.

Traps

- Order: `zero_grad` → forward → loss → `backward` → `step`.
- The batched loss must be reduced to a scalar before `backward()`.
- `nn.Softmax(dim=0)` works for a single vector; with batches use `dim=1` (classes axis).
- Dropout is *off* at test time (`model.eval()`).
- The output-layer gradient $(\mathbf{p} - \mathbf{e})\mathbf{z}^\top$ has the same form as multiclass LR's.

9.5 Practice

Practice

1. In the worked example, which entries of $\partial\mathcal{L}/\partial V$ are always 0 for this input, and why?
2. With ReLU instead of tanh in the worked example, what are $\mathbf{z}$ and $\partial\mathcal{L}/\partial\mathbf{a}$'s second entry?
3. A batch of 32 examples, 5 classes, hidden size 100, input 300. Give the shapes of the input, hidden activations, output probabilities and the loss.

Answers

1. Column 2 (multiplies $f_2 = 0$). 2. $\mathbf{a} = [0.7, -1]$, so $\mathbf{z} = [0.7, 0]$. $\text{ReLU}'(-1) = 0$, so the second entry is 0: no gradient flows to row 2 of V. 3. [32, 300], [32, 100], [32, 5], scalar.